21/04/2026

UADE

CINEFIND

Sistema de Calificación y Recomendación de Películas

Autores: Taffarel Zitelli Tomás – Project Manager
Etchepareborda Agustín – Documentador técnico
Santin Gigante Thiago Gabriel - Documentador técnico
Mendez Luciano Manuel – Desarrollador Backend
Ciampoli Juan Pablo - Desarrollador Backend
Farsetti Valentino – Analista Funcional
Equipo: Grupo 8
Carrera: Ingeniería en Informática
Institución: UADE
Materia: Programación / Ingeniería de Software
Fecha: Abril 2026

RESUMEN EJECUTIVO

Cinefind es un sistema de software desarrollado en Python cuyo objetivo es permitir a los usuarios calificar películas, gestionar información relacionada y facilitar la exploración de contenido cinematográfico.

El sistema surge como solución a la necesidad de organizar y analizar preferencias de usuarios en el ámbito del entretenimiento digital, permitiendo centralizar valoraciones y generar una base de datos estructurada.

El proyecto se encuentra actualmente en desarrollo y utiliza tecnologías como Python, estructuras de datos, y control de versiones con GitHub.

INTRODUCCIÓN

Contexto del proyecto

En la actualidad, el consumo de contenido audiovisual ha crecido significativamente, generando la necesidad de herramientas que permitan organizar, evaluar y recomendar películas de manera eficiente.

Importancia del sistema

La importancia del sistema Cinefind radica en que aborda un problema frecuente dentro del dominio del entretenimiento digital: la necesidad de registrar, organizar y consultar películas junto con sus valoraciones de manera estructurada. Aunque se trata de un proyecto académico, su utilidad representa un caso real de gestión de información, ya que permite almacenar títulos, géneros, puntajes y años de estreno, además de consultar estadísticas, buscar elementos concretos y visualizar películas destacadas. En términos de ingeniería de software, esto convierte al proyecto en una base válida para modelar un sistema de información pequeño pero escalable.

Desde una perspectiva formativa, el sistema también es importante porque materializa varios conceptos fundamentales del desarrollo de software: entrada y validación de datos, separación de responsabilidades mediante funciones, mantenimiento de consistencia entre estructuras de datos, procesamiento de colecciones, ordenamiento, búsqueda, cálculo de métricas y construcción de una interfaz básica de interacción por consola. En otras palabras, no solo resuelve una necesidad funcional, sino que además sirve como evidencia de aprendizaje técnico en Python y de aplicación de lógica algorítmica.

Asimismo, Cinefind posee valor como punto de partida para futuras mejoras. La existencia de funciones para agregar, actualizar, buscar, ordenar, calcular estadísticas, mostrar películas destacadas y validar correo electrónico demuestra que el sistema ya tiene una base funcional sobre la que podría evolucionar hacia una solución más completa, por ejemplo incorporando persistencia en archivos o base de datos, interfaz gráfica, usuarios registrados o integración con servicios externos de cine. Por eso, su importancia no solo está en lo que hace hoy, sino en su potencial de crecimiento como producto de software.

Alcance

El alcance actual del proyecto está centrado en la gestión básica de películas mediante una aplicación de consola. El sistema permite cargar nuevas películas, consultar las ya existentes, actualizar ciertos atributos, buscar registros concretos, obtener métricas agregadas y destacar películas con alto puntaje. Además, incluye un mecanismo de validación de correo electrónico como funcionalidad complementaria al cierre del programa. Todo esto se ejecuta en memoria durante la ejecución del script y se apoya en cuatro listas principales: películas, géneros, puntajes y años.

Dentro de ese alcance, el sistema sí cubre las siguientes capacidades: carga de películas nuevas con validación de nombre, género, puntaje y año; prevención de duplicados al agregar; modificación de género o puntaje de una película ya existente; cálculo del promedio general; ordenamiento por puntaje; búsqueda por nombre sin distinción entre mayúsculas y minúsculas; obtención de estadísticas como máximo, mínimo y promedio; generación de matrices para visualizar los datos; y listado de películas destacadas con puntaje mayor a 8.

Por otro lado, quedan fuera del alcance actual varias características que serían esperables en una versión más madura del sistema. No existe persistencia de datos entre ejecuciones, por lo que la información no se guarda de forma permanente. Tampoco hay autenticación de usuarios, manejo de múltiples perfiles, conexión con APIs de películas, interfaz gráfica, manejo de archivos externos ni arquitectura por capas. Además, la estructura de datos empleada mediante listas paralelas es funcional para una escala pequeña, pero no es la más robusta para una evolución compleja del producto. Por lo tanto, el alcance debe entenderse como el de un prototipo funcional académico, orientado a demostrar lógica de programación y manipulación de datos.

OBJETIVOS

Objetivo general

Desarrollar un sistema en Python que permita gestionar, calificar y consultar información de películas de manera eficiente.

Objetivos específicos

- Implementar estructuras de datos para almacenar películas
- Permitir ingreso de calificaciones
- Desarrollar funcionalidades de consulta
- Mantener código modular y escalable
- Aplicar buenas prácticas de programación

DESCRIPCIÓN DEL SISTEMA

¿Qué hace el sistema?

Cinefind permite:

- Registrar películas
- Asignar calificaciones
- Consultar información almacenada

Usuarios involucrados

- Usuario final (quien califica películas)
- Administrador (gestiona datos, en futuras versiones)

Casos de uso principales

- Registrar una película
- Calificar una película
- Consultar películas
- Visualizar puntuaciones

REQUERIMIENTOS

Requerimientos funcionales

- El sistema debe permitir agregar películas
- El sistema debe permitir calificar películas
- El sistema debe almacenar información
- El sistema debe mostrar datos al usuario

Requerimientos no funcionales

- Usabilidad: interfaz simple
- Mantenibilidad: código modular
- Escalabilidad: posibilidad de agregar nuevas funciones
- Rendimiento: respuesta rápida en operaciones básicas

ARQUITECTURA DEL SISTEMA

La arquitectura actual del sistema puede describirse como una arquitectura monolítica y procedural de consola. Todo el comportamiento del sistema está centralizado en un único archivo fuente, dentro del cual se agrupan funciones auxiliares de validación, funciones de negocio, funciones de visualización y la función principal “main()”, que orquesta el ciclo de vida del programa. Esta decisión es coherente con el contexto de un trabajo de algoritmos y estructuras de datos, donde la prioridad suele estar en la lógica de resolución antes que en una distribución formal por módulos o capas.

En términos de justificación, esta arquitectura resulta adecuada para una primera etapa del proyecto porque minimiza complejidad accidental, favorece la comprensión del flujo y permite demostrar operaciones fundamentales de programación.

TECNOLOGÍAS UTILIZADAS

Lenguajes

- Python

Herramientas

- Git
- GitHub - https://github.com/TomasTaff/Cinefind-Grupo8
- Visual Studio Code

Librerías

- Librerías estándar de Python

IMPLEMENTACIÓN

Tipo de arquitectura

El sistema Cinefind presenta una arquitectura monolítica de tipo procedural, implementada en un único archivo de código fuente. Este enfoque se basa en la ejecución secuencial de instrucciones y en la organización del sistema mediante funciones independientes que encapsulan distintas responsabilidades.

A nivel conceptual, la arquitectura puede interpretarse como una separación lógica en tres grandes componentes:

- Entrada y validación de datos
- Lógica de negocio
- Presentación en consola

Si bien no existe una separación física en módulos o capas (como en arquitecturas más avanzadas tipo MVC o cliente-servidor), sí hay una clara organización funcional que permite comprender el flujo del sistema.

Estructura interna del sistema

El sistema se organiza en torno a un flujo principal que controla la ejecución general mediante un ciclo iterativo. Este flujo central coordina la interacción con el usuario a través de un menú, desde el cual se accede a las distintas funcionalidades disponibles.

Internamente, el sistema maneja la información utilizando una estructura basada en listas paralelas. Esto implica que cada entidad lógica “película” no está representada como un objeto único, sino como un conjunto de atributos distribuidos en diferentes listas (nombre, género, puntaje y año), donde la posición dentro de cada lista actúa como identificador implícito.

Este diseño permite una implementación simple y directa, aunque requiere mantener sincronización estricta entre todas las estructuras de datos para evitar inconsistencias.

Gestión de datos

La gestión de datos se realiza completamente en memoria, lo que implica que toda la información cargada durante la ejecución se pierde al finalizar el programa. No existe persistencia en archivos ni en bases de datos.

El uso de listas paralelas implica las siguientes características:

- Simplicidad de implementación
- Acceso directo mediante índices
- Bajo costo computacional en operaciones básicas
- Alta dependencia entre estructuras

Como consecuencia, cualquier operación que modifique el orden o contenido de los datos debe aplicarse de manera coordinada en todas las listas para mantener la integridad del sistema.

Validación y control de errores

El sistema incorpora mecanismos de validación en la entrada de datos para garantizar que la información ingresada cumpla con ciertas condiciones mínimas. Estas validaciones incluyen:

- Verificación de campos no vacíos
- Restricción de rangos numéricos
- Validación de formato (por ejemplo, correo electrónico)

Este enfoque mejora la robustez del sistema y evita errores en etapas posteriores del procesamiento.

Justificación de la arquitectura

La elección de una arquitectura procedural monolítica se justifica por el contexto académico del proyecto y por los objetivos de aprendizaje asociados. Este enfoque permite:

- Focalizar en la lógica de programación
- Reducir la complejidad estructural
- Facilitar la comprensión del flujo de ejecución
- Implementar rápidamente funcionalidades básicas

Sin embargo, desde una perspectiva profesional, esta arquitectura presenta limitaciones en términos de escalabilidad, mantenibilidad y reutilización del código.

Funciones del sistema

Funciones de entrada y validación

def pedir_nombre()

```python
def pedir_nombre():
    nombre = input("Nombre de la película: ")
    while nombre == "":
        print("El nombre no puede estar vacío.")
        nombre = input("Nombre de la película: ")
    return nombre
```

Solicita el nombre de la película y valida que no sea un campo vacío.

def pedir_genero():

```python
def pedir_genero():
    genero = input("Género: ")
    while genero == "":
        print("El género no puede estar vacío.")
        genero = input("Género: ")
    return genero
```

Solicita el género de la película y valida que no sea un campo vacío.

def pedir_puntaje():

```python
def pedir_puntaje():
    puntaje_str = input("Puntaje (0 a 10): ")
    while not re.fullmatch(r"^\d{1,2}$", puntaje_str) or float(puntaje_str) < 0 or float(puntaje_str) > 10:
        print("Puntaje inválido.")
        puntaje_str = input("Puntaje (0 a 10): ")

    puntaje = float(puntaje_str)
    return puntaje
```

Solicita un puntaje (0-10). Usa re.fullmatch para asegurar que sean números y verifica que el valor esté en el rango permitido.

def pedir_anio():

```python
def pedir_año():
    año_str = input("Año de estreno: ")
    while not re.fullmatch(r"^\d{4}$", año_str) or int(año_str) < 1888 or int(año_str) > 2025:
        print("Año inválido o fuera de rango. Ingrese 4 números (ej. 2001).")
        año_str = input("Año de estreno: ")
    año = int(año_str)
    return año
```

Solicita el año de estreno. Valida que sea un número de 4 dígitos entre 1888 y 2025 mediante expresiones regulares y comparaciones lógicas.

def pedir_mail():

```python
def pedir_mail():
    patron = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
    mail = input("Ingrese su mail: ")
    while not re.match(patron, mail):
        print("Formato de mail inválido (ejemplo: usuario@dominio.com).")
        mail = input("Ingrese su mail: ")
    return mail
```

Valida la entrada de un correo electrónico mediante una expresión regular que verifica la estructura estándar (usuario@dominio.com).

Funciones de gestión de películas

def existe_pelicula(nombre, peliculas):

```python
def existe_pelicula(nombre, peliculas):
    i = 0
    while i < len(peliculas):
        if re.fullmatch(nombre, peliculas[i], flags=re.IGNORECASE):
            return True
        i += 1
    return False
```

Recorre la lista de títulos para verificar si el nombre ingresado ya existe. Usa re.IGNORECASE para evitar duplicados por diferencias de mayúsculas.

def agregar_pelicula(peliculas, generos, puntajes, anios):

```python
def agregar_pelicula(peliculas, generos, puntajes, años):
    nombre = pedir_nombre()
    if existe_pelicula(nombre, peliculas):
        print("Esa película ya fue cargada.\n")
        return
    genero = pedir_genero()
    puntaje = pedir_puntaje()
    año = pedir_año()
    peliculas.append(nombre)
    generos.append(genero)
    puntajes.append(puntaje)
    años.append(año)
    print("Película agregada.\n")
```

Coordina la carga de una nueva película. Verifica existencia previa y actualiza las cuatro listas (nombres, géneros, puntajes, años).

def actualizar_pelicula(peliculas, generos, puntajes, anios):

```
def actualizar_pelicula(peliculas, generos, puntajes, años):
    if len(peliculas) == 0:
        print("No hay películas para actualizar.\n")
        return
    nombre = input("Nombre de la película a actualizar: ")
    pos = -1
    for i in range(len(peliculas)):
        if peliculas[i] == nombre:
            pos = i
    if pos == -1:
        print("Esa película no está en la lista.\n")
        return
    print("1. Cambiar género")
    print("2. Cambiar puntaje")
    opcion = input("Opción: ")
    if opcion == "1":
        nuevo_genero = pedir_genero()
        generos[pos] = nuevo_genero
        print("Género actualizado.\n")
    elif opcion == "2":
        nuevo_puntaje = pedir_puntaje()
        puntajes[pos] = nuevo_puntaje
        print("Puntaje actualizado.\n")
    else:
        print("Opción inválida.\n")
```

Busca una película por nombre y permite al usuario modificar específicamente su género o su puntaje.

Funciones de procesamiento y análisis

def promedio_puntajes(puntajes):

```
def promedio_puntajes(puntajes):
    if len(puntajes) == 0:
        print("No hay películas cargadas.\n")
        return

    total = reduce(lambda a, b: a + b, puntajes, 0)

    promedio = total / len(puntajes)
    print("Puntaje promedio general:", round(promedio, 2), "\n")
```

Calcula el promedio aritmético de todos los puntajes registrados utilizando la función de orden superior reduce.

def copiar_lista(lista):

```
def copiar_lista(lista):
    copia_lista=lista.copy()
    return copia_lista
```

Crea y retorna una copia superficial (shallow copy) de la lista ingresada.

def ordenar_por_puntaje(peliculas, generos, puntajes, anios):

```python
def ordenar_por_puntaje(peliculas, generos, puntajes, años):
    n = len(puntajes)
    for i in range(n - 1):
        for j in range(n - i - 1):
            if puntajes[j] < puntajes[j + 1]:
                # intercambio
                puntajes[j], puntajes[j+1] = puntajes[j+1], puntajes[j]
                peliculas[j], peliculas[j+1] = peliculas[j+1], peliculas[j]
                generos[j], generos[j+1] = generos[j+1], generos[j]
                años[j], años[j+1] = años[j+1], años[j]
    print("Películas ordenadas por puntaje.\n")
```

Aplica el algoritmo de Bubble Sort para ordenar las listas de forma descendente basándose en el puntaje, manteniendo la integridad de los datos en paralelo.

def buscar_pelicula(peliculas, generos, puntajes, anios):

```python
def buscar_pelicula(peliculas, generos, puntajes, años):
    nombre = input("Ingrese el nombre de la película: ")
    for i in range(len(peliculas)):
        if re.fullmatch(nombre, peliculas[i], flags=re.IGNORECASE):
            print("Película encontrada:")
            print("Película:", peliculas[i])
            print("Género:", generos[i])
            print("Puntaje:", puntajes[i])
            print("Año:", años[i])
            return
    print("No se encontró la película.\n")
```

Realiza una búsqueda por nombre y, de encontrar coincidencia, imprime toda la ficha técnica de la película (género, puntaje, año).

def estadisticas(puntajes):

```python
def estadisticas(puntajes):
    if len(puntajes) == 0:
        print("No hay películas cargadas.\n")
        return
    maximo = max(puntajes)
    minimo = min(puntajes)
    promedio = sum(puntajes) / len(puntajes)
    print("Puntaje máximo:", maximo)
    print("Puntaje mínimo:", minimo)
    print("Promedio:", round(promedio, 2), "\n")
```

Muestra un resumen estadístico rápido: puntaje más alto, más bajo y el promedio general.

Funciones de visualización

def matriz_peliculas(filas, columnas, peliculas):

```python
def matriz_peliculas(filas, columnas,peliculas):
    matriz = []
    indice = 0
    for i in range(filas):
        fila = []
        for j in range(columnas):
            if indice < len(peliculas):
                fila.append(peliculas[indice])
                indice += 1
            else:
                fila.append("VACIO")
        matriz.append(fila)
    return matriz
```

Organiza la lista de nombres de películas en una estructura bidimensional (matriz) de dimensiones dadas, rellenando con "VACIO" si sobran espacios.

def imprimir_matriz(matriz):

```python
def imprimir_matriz(matriz):
    for i in range(len(matriz)):
        for j in range(len(matriz[i])):
            print(matriz[i][j], end=" | ")
        print()
```

Recorre e imprime cualquier matriz en consola con un formato visual de rejilla (|).

def mostrar_pelucula_en_matriz(filas, columnas, peliculas, generos, puntajes, anios):

```python
def mostrar_pelucula_en_matriz(filas, columnas,peliculas,generos,puntajes,años):
    matriz = []
    indice = 0
    for i in range(filas):
        fila = []
        for j in range(columnas):
            if indice < len(peliculas):
                fila.append([peliculas[indice],generos[indice],puntajes[indice],años[indice]])
                indice += 1
            else:
                fila.append("VACIO")
        matriz.append(fila)
    return matriz
```

Genera una matriz donde cada celda contiene una lista con toda la información agrupada de cada película.

def mostrar_destacadas(peliculas, puntajes):

```python
def mostrar_destacadas(peliculas, puntajes):
    print("\n=== PELÍCULAS DESTACADAS ===")
    posiciones = list(range(len(puntajes)))
    pos_destacadas = list(filter(lambda i: puntajes[i] > 8, posiciones))
    if len(pos_destacadas) == 0:
        print("No hay películas destacadas aún.")
    else:
        for i in pos_destacadas:
            print(f"- {peliculas[i]}: {puntajes[i]} estrellas")
    print()
```

Filtra y muestra por consola los nombres de las películas cuya calificación es estrictamente superior a 8 puntos usando filter.

Funciones de control del sistema

def menu():

```python
def menu():X
    print("=== MENÚ CINEFIND ===")
    print("1. Agregar película")
    print("2. Mostrar películas")
    print("3. Actualizar película")
    print("4. Calcular promedio")
    print("5. Ordenar por puntaje")
    print("6. Buscar película")
    print("7. Ver estadísticas")
    print("8. Ver peliculas destacadas")
    print("0. Salir")
    opcion = input("Opción: ")
    while opcion not in ["0","1","2","3","4","5","6","7","8"]:
        print("Opción inválida.")
        opcion = input("Opción: ")
    return opcion
```

Muestra la interfaz de opciones al usuario y valida que la entrada sea una opción válida.

RESULTADOS

Qué se logró

- Sistema funcional básico
- Estructura clara y extensible
- Código organizado

Evaluación del sistema

El sistema es:
- Correcto en funcionalidades principales
- Mejorable en interfaz y escalabilidad

CONCLUSIÓN

Aprendizajes

- Aplicación práctica de Python
- Diseño de sistemas modulares
- Uso de control de versiones
- Importancia de la organización del código

Mejoras futuras

- Implementar base de datos
- Integrar API de películas
- Añadir interfaz gráfica
- Implementar sistema de recomendaciones
- Gestión de usuarios

BIBLIOGRAFÍA

- SWEBOK v3 – Guide to the Software Engineering Body of Knowledge
- Documentación oficial de Python
- GitHub – Repositorio del proyecto Cinefind
- Material de cátedra